LOGISTIXPRESS · CENTRO DE DISTRIBUCIÓN

Royal Express

Memoria técnica

Arquitectura, modelo de datos e implementación

Documento de cierre técnico: decisiones de arquitectura, MVC, CRUD, RBAC, algoritmos de rutas, recepción, picking/packing, validación y despliegue OCI.

Producto	Royal Express · LogistiXpress
Documento	Memoria técnica
Versión	1.0.0
Fecha	Noviembre de 2026
Plataforma	FastAPI · PostgreSQL 16 · Docker/Podman
Estado	Implementación operativa · versión 1.0.0

Documento de Royal Express para LogistiXpress.
Uso interno del centro de distribución.

Índice general

1 Resumen ejecutivo **3**

2 Alcance y método **3**
2.1 Fuente de requerimientos . . . 3
2.2 Método de trabajo . . . 3
2.3 Convenciones del documento . . . 3

3 Arquitectura general **4**
3.1 Vista de componentes . . . 4
3.2 Flujo de una petición . . . 4
3.3 MVC en capas . . . 4
3.4 Estructura del repositorio . . . 5

4 Modelo de datos **6**
4.1 Esquemas . . . 6
4.2 Tablas principales . . . 6
4.2.1 Almacén . . . 6
4.2.2 Operaciones . . . 6
4.2.3 Seguridad y auditoría . . . 7
4.3 Relaciones clave . . . 7
4.4 Reglas de integridad . . . 7
4.5 Trigger de stock . . . 7
4.6 Secuencias . . . 8
4.7 Vistas . . . 8

5 Capa backend **8**
5.1 Organización . . . 8
5.2 Pool y rol de base de datos . . . 8
5.3 Inyección de dependencias . . . 9
5.4 Validación . . . 9
5.5 Autenticación . . . 9
5.6 Evidencias . . . 9
5.7 Reportes . . . 9

6 Frontend **11**
6.1 Filosofía . . . 11
6.2 Navegación . . . 11
6.3 Comunicación con la API . . . 11
6.4 Diseño visual . . . 11

7 Módulos funcionales **11**
7.1 Maestros y pedidos . . . 11
7.2 Recepción y putaway . . . 12
7.3 Picking y packing . . . 12
7.4 Rutas, despacho, entrega y rastreo . . . 12
7.5 Inventario, auditoría y panel . . . 12

8 Algoritmos **12**
8.1 Distancia . . . 12
8.2 Consolidación . . . 12
8.3 Optimización . . . 13

8.4 Métricas . . . 13
8.5 Picking . . . 13
8.6 Etiquetas . . . 13

9 Seguridad 13
9.1 Modelo de confianza . . . 13
9.2 Amenazas y mitigaciones . . . 13
9.3 Roles y GRANT . . . 14
9.4 Contraseñas y tokens . . . 14

10 Despliegue 14
10.1 Imagen de API . . . 14
10.2 Imagen de base de datos . . . 14
10.3 Compose . . . 14
10.4 Operación . . . 15
10.5 Por qué OCI y no AppImage . . . 15

11 Pruebas y validación 15
11.1 Pruebas unitarias . . . 15
11.2 Validación SQL . . . 15
11.3 Validación E2E . . . 15
11.4 Validación de interfaz . . . 16
11.5 OpenAPI . . . 16

12 Decisiones y trade-offs 16

13 Limitaciones y trabajo futuro 17

14 Conclusiones 17

15 Referencias 17

16 Apéndice A. Endpoints principales 18

17 Apéndice B. Matriz CRUD resumida 19

18 Apéndice C. Comandos de operación 19

19 Apéndice D. Credenciales iniciales 20

1 Resumen ejecutivo

Royal Express es el sistema de gestión de almacén y reparto desarrollado para el centro de distribución LogistiXpress. El proyecto convirtió los requerimientos del cliente en un producto operable, con una base de datos relacional, una API FastAPI y un panel web local que cubre el ciclo completo:

- Maestros de clientes, productos, ubicaciones, lotes, proveedores y pedidos.
- Recepción de mercancía con órdenes de compra, lotes, caducidad, condición e incidencias.
- Picking por pasillo, surtido guiado, empaque, etiquetas y control de calidad.
- Consolidación de rutas, despacho con escaneo, entrega con evidencia y rastreo por guía.
- Inventario en vivo, traslados internos, zonas especiales, conteos cíclicos.
- Panel de control, reportes PDF/XLSX y auditoría de actividades.

La implementación se organizó como una aplicación MVC en capas: PostgreSQL y el dominio representan el modelo; FastAPI actúa como controlador; la SPA funciona como vista. El resultado es una implementación reproducible y documentada, con límites explícitos para producción.

Resultado: 10 pruebas unitarias aprobadas, validación end-to-end con PostgreSQL real, interfaz validada con jsdom, 56 rutas OpenAPI y un flujo completo desde la recepción hasta la entrega evidenciada.

2 Alcance y método

2.1 Fuente de requerimientos

El punto de partida fue la *Solicitud de desarrollo de software para centro de distribución y logística "LogistiXpress"*. El documento se dividió en siete bloques funcionales:

1. Recepción de mercancía y descarga en andenes.
2. Acomodo y ubicación en racks.
3. Preparación de pedidos: picking y packing.
4. Embarques, rutas y control de flota.
5. Panel de control y auditorías.
6. Seguridad, roles y accesos.
7. Acceso y facilidad de uso.

La entrega cubre los siete bloques a nivel operativo, con decisiones de alcance en hardware de escaneo, PEPS/FIFO estricto, rechazo de QC y auditorías por área.

2.2 Método de trabajo

El desarrollo siguió cuatro principios:

- **Vertical slices:** cada módulo debía poder operarse de punta a punta, no solo como tablas o endpoints aislados.
- **Una sola fuente de verdad:** PostgreSQL concentra el estado; el backend no mantiene estado crítico en memoria.
- **Defensa en profundidad:** el rol se valida en FastAPI y también en PostgreSQL mediante `SET LOCAL ROLE`.
- **Reproducibilidad:** carga inicial, validación E2E y empaquetado OCI permiten reconstruir el entorno desde cero.

2.3 Convenciones del documento

- **Énfasis:** los nombres de archivos, tablas, endpoints y comandos se presentan en `monoespaciado`.
- **Estado:** los estados del sistema se escriben en MAYÚSCULAS (`LISTO`, `EN_RUTA`, `VALIDADO`).

- **Alcance:** toda afirmación se refiere a la versión 1.0 de Royal Express.

3 Arquitectura general

3.1 Vista de componentes

```
+-------------------------------------------------------------+
| Navegador * SPA HTML/CSS/JS                                 |
|  - sesión, router hash, modales, tablas, mapas, etiquetas   |
+------------------------------+------------------------------+
                               | fetch /api (Bearer)
                               v
+-------------------------------------------------------------+
| FastAPI                                                     |
|  routers -> servicios -> repositorios                        |
|  auth HMAC * RBAC * validación Pydantic * errores en español|
+------------------------------+------------------------------+
                               | psycopg 3 + pool
                               v
+-------------------------------------------------------------+
| PostgreSQL 16                                               |
|  almacen * operaciones * seguridad * auditoria               |
|  roles rol_admin / rol_operador / rol_auditor               |
|  trigger de stock * vistas * bitácora                       |
+-------------------------------------------------------------+
```

El frontend se sirve desde el mismo proceso FastAPI. No hay un servidor de frontend separado ni una CDN: la aplicación funciona en loopback y puede empaquetarse como imagen OCI.

3.2 Flujo de una petición

1. El navegador envía `fetch` con el token Bearer.
2. FastAPI valida el token y el rol.
3. El router valida el cuerpo con Pydantic.
4. El servicio ejecuta reglas de negocio.
5. El repositorio abre una transacción con psycopg.
6. La transacción fija el rol de PostgreSQL con `SET LOCAL ROLE`.
7. La consulta se ejecuta con parámetros, nunca con concatenación.
8. Al cerrar, la transacción hace commit o rollback.
9. El router serializa la respuesta y el panel la renderiza.

3.3 MVC en capas

Capa MVC	Implementación	Responsabilidad
Modelo	PostgreSQL + `app/modelos/dominio.py` + esquemas Pydantic	Datos, relaciones, estados, reglas de dominio
Vista	`royal-front/`	Presentación, navegación, formularios, mapas y etiquetas
Controlador	`app/routers/` + `app/servicios/`	Endpoints, permisos, validación y coordinación
Persistencia	`app/repositorios/`	SQL parametrizado y transacciones
Infraestructura	`compose.yaml`, `Dockerfile`, scripts	Ejecución local, OCI y datos iniciales

La separación evita que el frontend dependa de SQL y que los routers contengan lógica de base de datos. Los servicios concentran las reglas que no pertenecen ni a la vista ni al repositorio.

3.4 Estructura del repositorio

```
royal-front/
  index.html
  styles.css
  app.js
  js/core.js
  js/pages.js
  js/operaciones.js

royal-back/
  main.py
  app/
    main.py
    config.py
    db.py
    seguridad.py
    dependencias.py
    errores.py
    modelos/
    repositorios/
    servicios/
    routers/
  db/
    01_esquema.sql
    02_semilla.sql
    Dockerfile
  scripts/
  tests/
  Dockerfile
  requirements.txt

compose.yaml
bruno/
docs/
```

4 Modelo de datos

4.1 Esquemas

El modelo se dividió en cuatro esquemas para separar responsabilidades:

Esquema	Contenido
`almacen`	Productos, ubicaciones, stock, lotes, movimientos, traslados
`operaciones`	Clientes, proveedores, OC, recepción, pedidos, picking, empaque, rutas, entregas y rastreo
`seguridad`	Usuarios y bitácora
`auditoria`	Conteos cíclicos y métricas de rotación

4.2 Tablas principales

4.2.1 Almacén

Tabla	Propósito
`almacen.ubicaciones`	Pasillo, rack, nivel, orden de picking, tipo de zona y estado
`almacen.productos`	SKU, nombre, mínimo, peso y volumen unitarios
`almacen.stock`	Existencia por producto y ubicación
`almacen.lotes`	Lote, caducidad, cantidad y condición
`almacen.movimientos`	Entradas y salidas inmutables
`almacen.traslados`	Reubicación entre posiciones

4.2.2 Operaciones

Tabla	Propósito
`clientes`	Código, nombre, tipo, zona, dirección y coordenadas
`proveedores`	Catálogo de proveedores
`ordenes_compra`	Cita/OC con proveedor, fecha, hora, andén y estado
`orden_compra_lineas`	Producto y cantidades solicitadas/recibidas
`recepciones`	Cabecera de una recepción
`recepcion_lineas`	Lote, caducidad, condición y ubicación recibida
`incidencias_recepcion`	Descripción, foto, responsable y comentario
`pedidos`	Pedido con cliente, estado, prioridad y totales
`pedido_detalle`	Líneas de producto del pedido
`picking_olas`	Ola de surtido
`picking_tareas`	Tarea por pedido/producto/ubicación
`empaques`	Cabecera de empaque y validación QC
`bultos`	Paquete con guía, peso, volumen y estado
`conductores`	Catálogo de conductores
`vehiculos`	Placas, modelo y capacidades
`rutas`	Cabecera de ruta y métricas
`ruta_paradas`	Secuencia, coordenadas, ETA y estado
`ruta_bultos`	Bultos asignados a una parada
`entregas`	Resultado, firma, foto, motivo y reprogramación
`eventos_rastreo`	Línea de tiempo por bulto

4.2.3 Seguridad y auditoría

Tabla	Propósito
`seguridad.usuarios`	Cuenta, hash, rol, puesto y estado
`seguridad.bitacora`	Registro inmutable de acciones
`auditoria.conteos_ciclicos`	Físico vs sistema y diferencia

4.3 Relaciones clave

```
clientes 1 --- N pedidos 1 --- N pedido_detalle N --- 1 productos
pedidos 1 --- N picking_tareas
pedidos 1 --- N empaques 1 --- N bultos
pedidos 1 --- N ruta_paradas N --- 1 rutas
ruta_paradas 1 --- N ruta_bultos N --- 1 bultos
ruta_paradas 1 --- 0..1 entregas
bultos 1 --- N eventos_rastreo
proveedores 1 --- N ordenes_compra 1 --- N orden_compra_lineas
ordenes_compra 1 --- N recepciones 1 --- N recepcion_lineas
```

4.4 Reglas de integridad

- **Guía única:** `bultos.guia` no se repite.
- **Folios únicos:** rutas, pedidos, OC, olas, empaques y recepciones.
- **Capacidad no forzada en SQL:** el planificador valida peso/volumen antes de persistir.
- **Estados acotados:** cada tabla usa `CHECK` para impedir transiciones inválidas.
- **Borrado lógico:** maestros y usuarios usan `activo`; las operaciones históricas no se borran.
- **Inmutabilidad:** movimientos, bitácora, entregas y eventos no tienen endpoints de edición o borrado.

4.5 Trigger de stock

El trigger `almacen.fn_actualizar_stock_tiempo_real` se ejecuta después de insertar un movimiento. Es `SECURITY DEFINER` y fija `search_path`, de modo que el operador puede registrar movimientos sin tener `UPDATE` directo sobre `stock`:

```
CREATE OR REPLACE FUNCTION almacen.fn_actualizar_stock_tiempo_real()
RETURNS TRIGGER
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = almacen, pg_temp
AS $$
BEGIN
    IF NEW.tipo = 'ENTRADA' THEN
        INSERT INTO almacen.stock (producto_id, ubicacion_id, cantidad)
        VALUES (NEW.producto_id, NEW.ubicacion_id, NEW.cantidad)
        ON CONFLICT (producto_id, ubicacion_id)
        DO UPDATE SET cantidad = almacen.stock.cantidad + EXCLUDED.cantidad;
    ELSIF NEW.tipo = 'SALIDA' THEN
        UPDATE almacen.stock
        SET cantidad = cantidad - NEW.cantidad
        WHERE producto_id = NEW.producto_id
          AND ubicacion_id = NEW.ubicacion_id
          AND cantidad >= NEW.cantidad;
        IF NOT FOUND THEN
            RAISE EXCEPTION 'Stock insuficiente para el producto % en la ubicación %',
                NEW.producto_id, NEW.ubicacion_id;
        END IF;
```

```
    END IF;
    RETURN NEW;
END;
$$;
```

4.6 Secuencias

Secuencia	Uso
`folio_pedido_seq`	Pedidos `PED-AAAA-####`
`folio_oc_seq`	Órdenes de compra `OC-AAAA-####`
`folio_ola_seq`	Olas `OLA-AAAAMMDD-####`
`folio_empaque_seq`	Empaques `EMP-AAAAMMDD-####`
`folio_guia_seq`	Guías `LX-#####`
`folio_ruta_seq`	Rutas `R-AAAAMMDD-####`

4.7 Vistas

Vista	Uso
`operaciones.v_ruta_picking`	Ruta de picking por ubicación y orden
`auditoria.v_metricas_rotacion`	Entradas/salidas por producto
`operaciones.v_resumen_rutas`	Resumen de rutas
`operaciones.v_entregas_fallidas`	Entregas con falla

5 Capa backend

5.1 Organización

Directorio	Responsabilidad
`app/routers/`	Endpoints, dependencias de rol y respuesta HTTP
`app/servicios/`	Reglas de negocio, planificación, render, reportes, evidencias
`app/repositorios/`	SQL y transacciones
`app/modelos/dominio.py`	Entidades puras del planificador
`app/modelos/esquemas*.py`	DTOs Pydantic de entrada/salida
`app/config.py`	DSN, secreto, evidencias y parámetros de ruta
`app/db.py`	Pool de conexiones y `SET LOCAL ROLE`
`app/seguridad.py`	Hash, tokens, roles y dependencias
`app/errores.py`	Error de negocio con traducción HTTP

5.2 Pool y rol de base de datos

El pool se crea de forma perezosa y cada transacción fija el rol correspondiente:

```
@contextmanager
def cursor(self, rol: str | None = None):
    if rol is not None and rol not in ROLES_BD_VALIDOS:
        raise ValueError(f"Rol de base de datos inválido: {rol}")
    self._abrir()
    with self._pool.connection() as conexion:
```

```
        with conexion.cursor() as cur:
            if rol:
                cur.execute(f"SET LOCAL ROLE {rol}")
            yield cur
```

`SET LOCAL ROLE` tiene dos efectos:

- Aplica los `GRANT` de PostgreSQL, no solo los permisos de FastAPI.
- Se revierte al cerrar la transacción, porque es local.

5.3 Inyección de dependencias

FastAPI inyecta:

- `usuario_actual` → valida el token.
- `requiere_roles("admin", "operador")` → protege el endpoint.
- `obtener_bd()` → pool compartido.
- Repositorios concretos con el rol de la sesión.

5.4 Validación

Pydantic valida:

- Tipos, longitudes y rangos.
- Literales de estado, zona, condición, prioridad y rol.
- Cuerpos JSON de movimientos, entregas, recepción, picking y usuarios.
- Respuestas tipadas en OpenAPI.

Los errores de negocio se lanzan como `ErrorNegocio` y se traducen a respuestas en español:

Código	Uso
400	Datos inválidos o regla de negocio
403	Rol sin permiso
404	Recurso inexistente
409	Conflicto de estado, stock, unicidad o validación de escaneo
503	PostgreSQL no disponible

5.5 Autenticación

- Contraseñas con PBKDF2-HMAC-SHA256 y 200 000 iteraciones.
- Token HMAC-SHA256 firmado con `ROYAL_SECRETO`.
- Vigencia de 12 horas por defecto.
- El token incluye id, nombre, usuario, correo y rol.
- `GET /api/auth/me` valida que la cuenta siga activa.
- No se usan cookies para la API; el token va en `Authorization: Bearer`.

5.6 Evidencias

- Firma y foto se reciben como data URL base64.
- Se validan MIME, firma mágica y tamaño máximo.
- Se guardan bajo `data/evidencias/<folio>/<parada>/`.
- La descarga exige autenticación.
- La resolución de rutas impide path traversal.

5.7 Reportes

- PDF con matplotlib y `PdfPages`.
- XLSX con openpyxl.
- Generación protegida por un lock compartido con el renderizador de mapas.

- Tipos: inventario, movimientos, rutas, entregas, auditoría, rotación y usuarios.

6 Frontend

6.1 Filosofía

La SPA se escribió sin frameworks ni CDN para que la aplicación sea autocontenida y funcione offline en loopback. Se usan HTML5, CSS3 y JavaScript clásico.

Archivo	Responsabilidad
`index.html`	Shell, login y sprite de iconos SVG
`styles.css`	Tema oscuro con acento naranja/rojo
`js/core.js`	Estado, API, sesión, router, modales, helpers
`js/pages.js`	Inicio, rutas, rastreo, flota, inventario, auditoría, usuarios, reportes, perfil
`js/operaciones.js`	Maestros, recepción y picking/packing
`app.js`	Arranque y estado de la API

6.2 Navegación

- Router por hash (`#/rutas`, `#/recepcion`, etc.).
- El rol oculta acciones, no solo secciones.
- El auditor ve tablas y etiquetas, pero no botones de operación.
- El menú lateral se desplaza si hay muchas secciones.
- La búsqueda superior abre el rastreo por guía.

6.3 Comunicación con la API

- `api()` agrega el token y parsea JSON.
- `apiBlob()` descarga mapas y evidencias con autenticación.
- Un `401` limpia la sesión y muestra el login.
- Los errores de validación se muestran en el modal correspondiente.
- Los mapas se convierten a `blob:` para no exponer la URL con token.

6.4 Diseño visual

- Tema oscuro con acento naranja/rojo, inspirado en operadores logísticos.
- Tarjetas KPI, paneles, tablas y modales consistentes.
- Botones grandes y etiquetas claras.
- Responsive para oficina, tablet y teléfono.
- Etiquetas imprimibles en fondo blanco.

7 Módulos funcionales

7.1 Maestros y pedidos

- CRUD de clientes, productos, ubicaciones, lotes, proveedores y pedidos.
- Alta/edición con validación de unicidad y baja lógica.
- Pedido con líneas y cálculo de peso/volumen desde los productos.
- Edición solo en BORRADOR; cancelación lógica.
- Bitácora de cada alta o cambio.

7.2 Recepción y putaway

1. Se crea una OC/cita con proveedor, fecha, hora y andén.
2. Se recibe por línea; el backend valida cantidades pendientes.
3. Se inserta un movimiento ENTRADA; el trigger actualiza stock.
4. Se crea o actualiza el lote.
5. Se registran incidencias con foto y responsable.
6. La OC cambia a RECIBIDA, EN_RECEPCION o CON_INCIDENCIA.
7. El endpoint de sugerencia propone una ubicación GENERAL con menor existencia.

7.3 Picking y packing

1. Se genera una ola con todos los pedidos BORRADOR.
2. Se valida stock en ubicaciones GENERAL.
3. Se crean tareas únicas por ola/pedido/producto.
4. Las tareas se ordenan por pasillo y `orden_picking`.
5. El surtidor confirma SKU y ubicación; un código incorrecto devuelve 409.
6. Se inserta un movimiento SALIDA y se marca la tarea PICKEADO.
7. Al completar el pedido, pasa a PICKING_COMPLETO.
8. Empaque: se crea empaque, bultos y guías; se registra ETIQUETADO.
9. QC: empaque VALIDADO y pedido LISTO.
10. El pedido LISTO entra al planificador de rutas.

7.4 Rutas, despacho, entrega y rastreo

- Consolidación por zona, tipo, prioridad, peso y volumen.
- Asignación de vehículo/conductor.
- Optimización de paradas.
- Mapa PNG y métricas.
- Despacho con validación de guías.
- Entrega con firma/foto o falla con motivo/reprogramación.
- Eventos de rastreo en cada transición.
- Rastreo por guía.

7.5 Inventario, auditoría y panel

- Stock en vivo, mínimos y vencimientos.
- Movimientos y traslados.
- Zonas especiales.
- Conteos cíclicos y mermas.
- Bitácora y rotación.
- KPIs de picking, empaque, mermas, estancados y productividad.

8 Algoritmos

8.1 Distancia

Se usa Haversine sobre latitud/longitud:

```
d = 2R * asin( sqrt( sin^2(dlat/2) + cos lat1 * cos lat2 * sin^2(dlon/2) ) )
```

Es suficiente para la versión 1.0 y evita depender de un proveedor de mapas.

8.2 Consolidación

1. Agrupar pedidos por zona.

2. Ordenar por prioridad de tipo de cliente, prioridad del pedido y peso descendente.
3. Abrir un viaje con un vehículo que soporte el primer pedido.
4. Asignar pedidos con first-fit decreciente mientras quepan en peso y volumen.
5. Repetir hasta agotar la zona.

8.3 Optimización

Se implementó vecino más cercano + 2-opt:

```
ruta = vecino_mas_cercano(paradas, deposito)
mientras haya mejora:
    para cada par de segmentos:
        invertir el segmento
        si la distancia total disminuye:
            aceptar
```

La distancia base se calcula con el orden de carga original. El ahorro se muestra al usuario.

8.4 Métricas

- Distancia optimizada.
- Distancia base.
- Ahorro porcentual.
- ETA = distancia/velocidad + paradas × minutos de entrega.

8.5 Picking

- Se busca stock GENERAL por producto.
- Se controla la cantidad ya asignada por ubicación.
- Se crea una tarea por pedido/producto.
- Se ordena por pasillo y `orden_picking`.
- La confirmación valida SKU y ubicación.
- La salida descuenta stock por trigger.

8.6 Etiquetas

- Se genera una etiqueta por bulto.
- Incluye cliente, dirección, guía, peso, volumen, pedido y total de bultos.
- El código es visual; no se implementó Code128 real.

9 Seguridad

9.1 Modelo de confianza

- La API y la base escuchan en loopback.
- El frontend no tiene acceso directo a PostgreSQL.
- Cada endpoint valida rol.
- Cada transacción fija el rol de base.
- No hay endpoints de borrado duro sobre operaciones.
- La bitácora registra las acciones críticas.

9.2 Amenazas y mitigaciones

Amenaza	Mitigación
SQL injection	Consultas parametrizadas con psycopg

Amenaza	Mitigación
XSS	Escape de datos en plantillas y `textContent` en errores
Escalada de rol	Guards en FastAPI + `SET LOCAL ROLE` + GRANT
Acceso a evidencia ajena	Endpoint autenticado y resolución de ruta segura
Path traversal	`Path.resolve()` + validación de raíz
Fuerza bruta	No implementada; pendiente para producción
Replay de token	Expiración de 12 h; sin revocación
CSRF	No se usan cookies de sesión
Exposición de puertos	Compose publica solo en 127.0.0.1

9.3 Roles y GRANT

- `rol_admin`: todos los privilegios.
- `rol_operador`: lectura general y escritura operativa.
- `rol_auditor`: lectura y conteos/bitácora.
- El usuario de conexión debe poder `SET LOCAL ROLE` (superusuario o miembro de roles).

9.4 Contraseñas y tokens

- PBKDF2-SHA256 con sal aleatoria.
- 200 000 iteraciones.
- Comparación en tiempo constante.
- Token firmado con HMAC; no se usa una librería JWT.
- El token no se guarda en la base.

10 Despliegue

10.1 Imagen de API

```
FROM python:3.12-slim
WORKDIR /app
COPY royal-back/requirements.txt ...
RUN pip install --no-cache-dir -r ...
COPY royal-back /app/royal-back
COPY royal-front /app/royal-front
WORKDIR /app/royal-back
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

10.2 Imagen de base de datos

La imagen de PostgreSQL copia el DDL y la semilla dentro de `/docker-entrypoint-initdb.d`. Esto evita depender de bind mounts desde el home del usuario y elimina problemas de permisos entre el host y el usuario `postgres` del contenedor.

```
FROM postgres:16-alpine
COPY 01_esquema.sql /docker-entrypoint-initdb.d/01_esquema.sql
COPY 02_semilla.sql /docker-entrypoint-initdb.d/02_semilla.sql
RUN chmod 0644 /docker-entrypoint-initdb.d/*.sql
```

10.3 Compose

- PostgreSQL y API publicados en `127.0.0.1`.
- Healthcheck de base que espera usuarios de la semilla.
- API dependiente de base sana.

- Volumen de datos y bind de evidencias con etiqueta SELinux `z`.
- Reinicio `unless-stopped`.

10.4 Operación

```
sudo docker compose down -v
sudo docker compose up --build
sudo docker compose logs -f api
```

10.5 Por qué OCI y no AppImage

La aplicación tiene dos procesos (API y PostgreSQL) y un volumen de datos. OCI resuelve el empaquetado, las dependencias y el ciclo de vida de ambos procesos sin requerir compilar Python ni PostgreSQL en el servidor de destino. Una AppImage o un binario con PyInstaller tendría que resolver igualmente la base de datos y sería menos reproducible.

11 Pruebas y validación

11.1 Pruebas unitarias

`pytest` valida:

- Planificador por zona y capacidad.
- Optimizador conserva paradas y no empeora.
- Error cuando un pedido excede la flota.
- Prioridad por tipo de cliente.
- Tokens y contraseñas.
- Evidencias (PNG, formato inválido, path traversal).
- Render PNG.

Resultado: **10 passed**.

11.2 Validación SQL

`pglast` parsea:

- `00_limpiar.sql`
- `01_esquema.sql`
- `02_semilla.sql`
- Consultas embebidas de los repositorios.

11.3 Validación E2E

`scripts/validar_e2e.py` levanta PostgreSQL 18 embebido, carga el esquema y la semilla, y prueba:

- Login de los tres roles.
- RBAC y rechazos 403.
- Generación de plan.
- Mapa PNG.
- Despacho con faltante y despacho válido.
- Entrega con firma y evidencia.
- Rastreo.
- Inventario y conteo.
- Reportes PDF/XLSX.
- CRUD de maestros.
- Recepción con lote y stock.
- Incidencia con foto.

- Ola de picking, SKU incorrecto, empaque, etiquetas y QC.
- Pedido reincorporado a ruta.
- Traslado interno.
- Edición de flota y usuarios.
- Panel extendido.

Resultados de la corrida de referencia:

```
Plan: 3 viajes * 11 pedidos * 27 bultos * 20.85% ahorro
Recepción OK * OC-2026-1000 * stock producto 2 198->208
Incidencia de recepción OK * foto PNG
Picking/empaque/QC OK * ola OLA-... * pedido ... * bultos 2
Pedido empacado enviado a ruta * R-...
Traslado interno OK * stock total 207->207
Panel extendido OK * picking 1 * empaque 0 * mermas 45 * productividad 1
```

11.4 Validación de interfaz

jsdom contra API/PostgreSQL reales:

- Login.
- 12 secciones.
- Maestros y formularios.
- Recepción y modal de recibo.
- Ola de picking, surtido, empaque, etiquetas y QC.
- Traslados, flota y usuarios.
- Sin errores de JavaScript.

11.5 OpenAPI

La aplicación expone **56 rutas** documentadas en `/docs`, agrupadas por autenticación, panel, rutas, entregas, rastreo, recepción, picking, inventario, maestros, flota, usuarios, auditoría y reportes.

12 Decisiones y trade-offs

Decisión	Motivo	Costo
SQL directo con psycopg	Transparencia, control de GRANT y consultas de agregación	Más código manual que un ORM
`SET LOCAL ROLE`	RBAC real en PostgreSQL	Requiere usuario con membresía/superusuario
Token HMAC propio	Sin dependencias, suficiente para la versión 1.0	Sin revocación ni estándar OAuth
Matplotlib para mapas	Offline, reproducible, sin API key	Mapa no interactivo ni con calles
Etiqueta con código visual	Sin dependencia de barcode	No escaneable
Tres roles + `puesto`	No rompe el paradigma acordado	Permisos finos por puesto no implementados
Sin borrado duro	Integridad y auditoría	No hay DELETE clásico en operaciones
Frontend sin CDN	Autocontenido y offline	Más CSS/JS propio
OCI con Compose	Reproducible y portable	Requiere Docker/Podman

13 Limitaciones y trabajo futuro

1. **Lector de códigos real:** integrar cámara/lector y Code128/QR.
2. **PEPS/FIFO estricto:** descontar lotes por caducidad y guardar lote en bulto.
3. **QC con rechazo y reempaque:** motivo, nueva etiqueta y trazabilidad.
4. **Auditorías por área:** supervisor, calidad y dirección con aprobación.
5. **Job de reprogramación:** pedidos REPROGRAMADO → LISTO en fecha.
6. **PWA/offline:** instalación y captura móvil.
7. **Object storage:** evidencias en S3/MinIO para múltiples instancias.
8. **Revocación de tokens:** invalidar sesión al desactivar cuenta.
9. **Migraciones:** reemplazar el reset de volumen por migraciones versionadas.
10. **Observabilidad:** métricas, trazas y alertas.
11. **Rate limiting:** protección de login y API.
12. **TLS:** exposición segura para tablets/teléfonos.

14 Conclusiones

Royal Express demuestra que el requerimiento puede implementarse con una arquitectura simple, auditable y reproducible. El valor técnico no está en la cantidad de tablas ni en la interfaz, sino en la coherencia del flujo:

- El stock se mueve por transacciones, no por edición directa.
- Las rutas se calculan y persisten con métricas.
- El despacho no confía en la vista: valida guías.
- La entrega deja evidencia.
- El rastreo reconstruye la historia del paquete.
- El rol se aplica en la API y en la base.
- Todo queda en bitácora.

Royal Express cubre el módulo 4 completo y el flujo integral de los siete bloques del documento. Las decisiones de alcance están identificadas y tienen una ruta clara de evolución.

15 Referencias

1. Royal Express. *Solicitud de desarrollo de software para centro de distribución y logística "LogistiXpress"*. Documento del cliente, 2026.
2. SharpNode. *HoneNode: Plano de gestión replicable para hosting de microservicios*. Referencia de estructura documental, 2026.
3. PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*.
4. FastAPI. *FastAPI Documentation*.
5. OWASP. *Application Security Verification Standard 4.0*.
6. W3C. *Web Content Accessibility Guidelines 2.2*.
7. Docker. *Compose Specification*.
8. Arch Linux. *Arch Wiki: Writing documentation*.

16 Apéndice A. Endpoints principales

Grupo	Rutas representativas
Autenticación	`POST /api/auth/login`, `GET /api/auth/me`, `POST /api/auth/logout`
Panel	`GET /api/panel/resumen`
Rutas	`GET/POST /api/rutas*`, `GET /api/rutas/{id}/mapa`, `POST /api/rutas/{id}/despachar`
Entregas	`POST /api/rutas/{id}/paradas/{parada}/entrega`, `GET /api/entregas/{id}/evidencia/{tipo}`
Rastreo	`GET /api/rastreo`, `GET /api/rastreo/{guia}`
Recepción	`GET/POST /api/recepcion/ordenes`, `POST /api/recepcion/ordenes/{id}/recibir`, `GET /api/recepcion/incidencias`
Picking	`GET /api/picking/pedidos`, `POST /api/picking/generar`, `POST /api/picking/tareas/{id}/confirmar`, `POST /api/picking/pedidos/{id}/empacar`, `POST /api/picking/pedidos/{id}/aprobar`
Maestros	`GET/POST/PATCH /api/maestros/clientes`, `productos`, `ubicaciones`, `lotes`, `proveedores`, `pedidos`
Inventario	`GET/POST /api/inventario/movimientos`, `GET/POST /api/inventario/traslados`, `GET /api/inventario/stock`, `GET /api/inventario/catalogo`
Flota	`GET /api/flota`, `POST/PATCH /api/flota/vehiculos`, `POST/PATCH /api/flota/conductores`
Usuarios	`GET/POST/PATCH /api/usuarios*`
Auditoría	`GET/POST /api/auditoria/conteos`, `GET /api/auditoria/bitacora`, `GET /api/auditoria/rotacion`
Reportes	`GET /api/reportes/exportar`

17 Apéndice B. Matriz CRUD resumida

Entidad	C	R	U	D
Usuarios	Sí	Sí	Sí	Baja lógica
Productos / ubicaciones	Sí	Sí	Sí	Baja lógica
Lotes	Sí	Sí	Sí	No
Stock	Movimiento	Sí	Trigger	No
Movimientos / traslados	Sí	Sí	No	No
Clientes / proveedores	Sí	Sí	Sí	Baja lógica
Pedidos	Sí	Sí	Sí (BORRADOR)	Cancelación
OC / recepciones	Sí	Sí	Recepción	Cancelación
Picking / empaque	Sí	Sí	Sí	No
Vehículos / conductores	Sí	Sí	Sí	Baja lógica
Rutas / paradas	Sí	Sí	Sí	Cancelación
Entregas / eventos	Sí	Sí	No	No
Conteos / bitácora	Sí	Sí	No	No

18 Apéndice C. Comandos de operación

```
# Operación completa
sudo docker compose down -v
sudo docker compose up --build

# Logs
sudo docker compose logs -f api
sudo docker compose logs -f db

# Pruebas
cd royal-back
source .venv/bin/activate
pip install -r requirements-dev.txt
pytest -q
python scripts/validar_e2e.py

# Validación SQL
python - <<'PY'
from pathlib import Path
from pglast import parse_sql
for f in ("db/01_esquema.sql", "db/02_semilla.sql"):
    parse_sql(Path(f).read_text(encoding="utf-8"))
print("SQL OK")
PY
```

19 Apéndice D. Credenciales iniciales

Rol	Usuario	Contraseña
Administración	`admin`	`admin123`
Operación	`operador`	`operador123`
Auditoría	`auditor`	`auditor123`

Advertencia. Estas credenciales son iniciales. Cambiar `ROYAL_SECRETO`, las contraseñas y el usuario de base antes de cualquier uso fuera del entorno controlado.